

Тема 12 – Линејни динамични АТД

Линејни динамични АТД:

1. стек,
2. опашка,
3. списък.

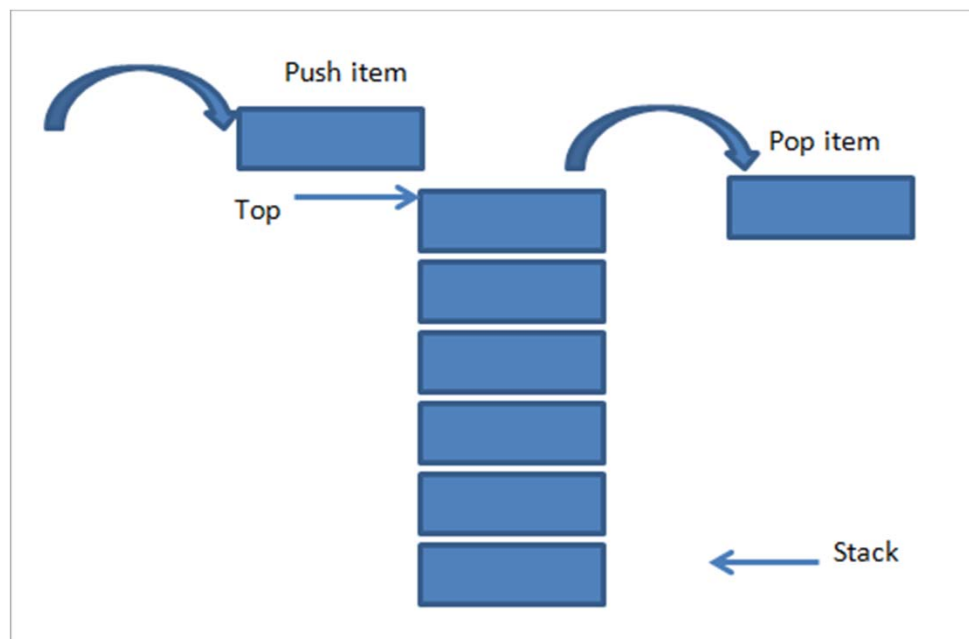
Основни алгоритми за работа с тях.

Тема 12 – Линејни динамични АТД

Стек в C++

Ако искаме да добавим друг елемент към стека, тогава го добавяме в горната му част, както е показано на фигурата . Тази операция за добавяне на елемент към стека се нарича „Push“.

От дясната страна е показана противоположната операция, т.е. премахваме елемент от стека. Това също се прави от същия край, т.е. от горната част на стека. Тази операция се нарича „Pop“.



Тема 12 – Линејни динамични АТД

Както е показано на горната фигура, виждаме, че „Push“ и „Pop“ се извршват от един и същи край, който се нарича „върх на стека“. Това прави стека да следва LIFO реда.

Първоначално, когато няма елементи в стека, горната част на стека е зададена на -1.

Когато добавим елемент към стека, горната част на стека се увеличава с 1, което показва, че елементът е добавен.

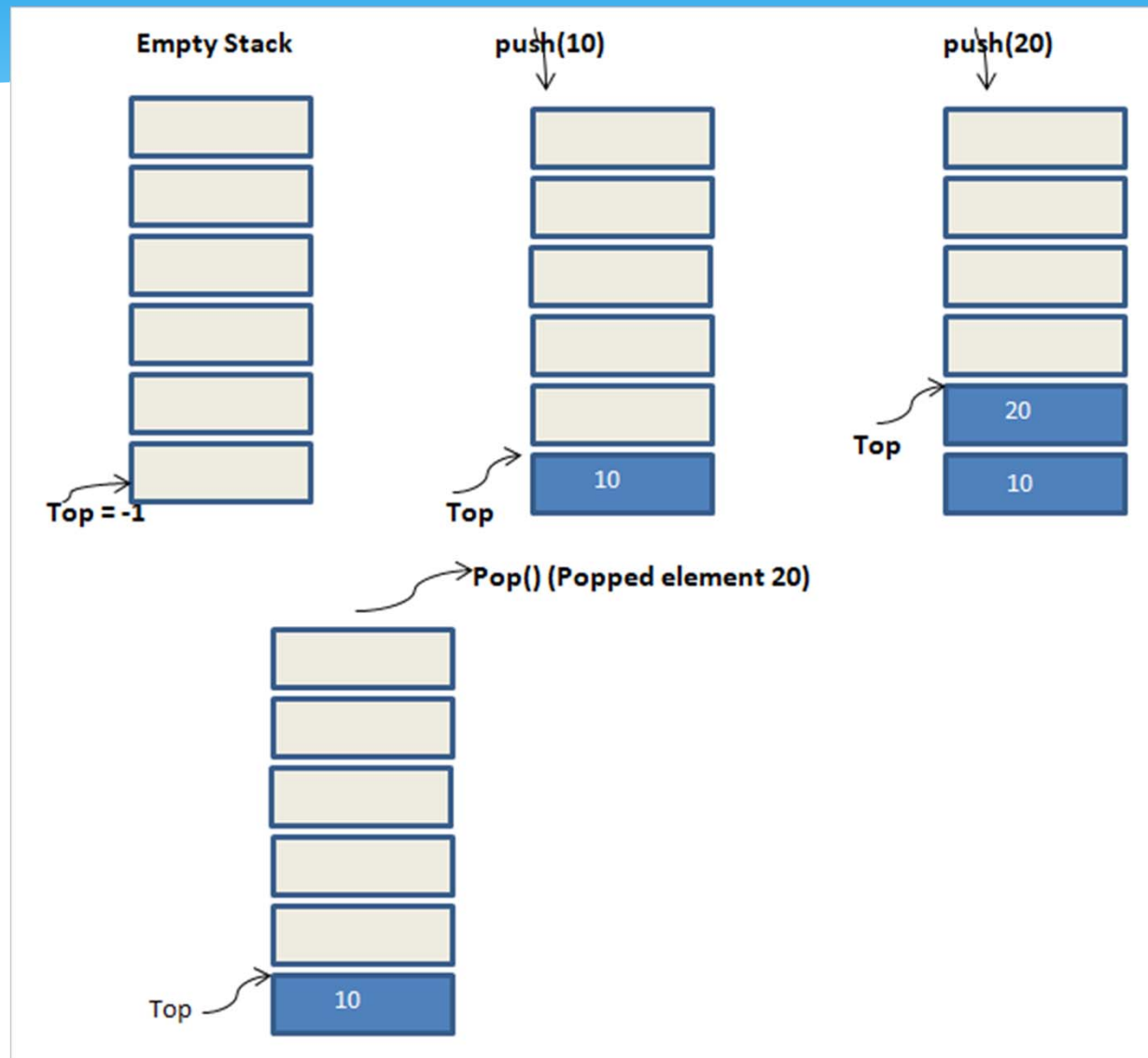
За разлика от това, горната част на стека се намалява с 1, когато даден елемент излезе от стека.

Тема 12 – Линејни динамични АТД

Основни операции, които се поддържат от стека :

- * **push** – Добавя елемент в стека.
- * **pop** – Премахва или изважда елемент от стека.
- * **peek** – Получава горния елемент на стека, но не го премахва.
- * **isFull** – Тества дали стекът е пълен.
- * **isEmpty** – Тества дали стекът е празен.

Тема 12 – Лінійні динамічні АТД



Тема 12 – Линејни динамични АТД

Фигурата по-горе показва последователноста от операции, които се изпълняват върху стека. Първоначално стекът е празен. За празен стек горната част на стека е зададена на -1.

След това избутваме елемент със стойност 10 в стека. Виждаме, че горната част на стека сега сочи към елемент 10.

След това извършваме друга операция за избутване с елемент със стойност 20, в резултат на което горната част на стека вече сочи към 20.

След това в последната фигура извършваме операция pop (). В резултат на операцията за изскачане елементът, посочен в горната част на стека, се премахва от стека. Следователно на фигурата виждаме, че елемент със стойност 20 е премахнат от стека. Така горната част на стека сега сочи към 10.

По този начин можем лесно да различим LIFO подхода, използван от стека.

Тема 12 – Линејни динамични АТД

Приложение на стек с масив:

```
#include <iostream>
```

```
using namespace std;
```

```
int stack[100], n=100, top=-1;
```

```
void push(int val) {
```

```
    if(top>=n-1)
```

```
        cout<<"Stack Overflow"<<endl;
```

```
    else {
```

```
        top++;
```

```
        stack[top]=val; }
```

```
}
```

```
void pop() {
```

```
    if(top<=-1)
```

```
        cout<<"Stack Underflow"<<endl;
```

```
    else {
```

```
        cout<<"The popped is "<< stack[top] <<endl;
```

```
        top--; }
```

```
}
```

```
void display()
```

```
{
```

```
    if(top>=0)
```

```
{
```

```
    cout<<"Stack elements are:";
```

```
    for(int i=top; i>=0; i--)
```

```
        cout<<stack[i]<<" ";
```

```
    cout<<endl;
```

```
}
```

```
else
```

```
    cout<<"Stack is empty";
```

```
}
```

Тема 12 – Линејни динамични АТД

```
int main() {
    int ch, val;
    cout<<"1) Push in stack"<<endl;
    cout<<"2) Pop from stack"<<endl;
    cout<<"3) Display stack"<<endl;
    cout<<"4) Exit"<<endl;
    do {
        cout<<"Enter choice: "<<endl;
        cin>>ch;
        switch(ch) {
            case 1: {
                cout<<"Enter value to be pushed:"<<endl;
                cin>>val;
                push(val);
                break;
            }
            case 2: {
                pop();
                break;
            }
            case 3: {
                display();
                break;
            }
            case 4: {
                cout<<"Exit"<<endl;
                break;
            }
            default: {
                cout<<"Invalid Choice"<<endl;
            }
        }
    }while(ch!=4);
    return 0;
}
```


Тема 12 – Линејни динамични АД

Приложение на стек със свързан списък:

```
#include <iostream>
using namespace std;
//Structure of the Node
struct Node
{
int data;
Node *link;
};
// top pointer to keep track of the top of the stack
Node *top = NULL;

//Function to check if stack is empty or not
bool isempty()
{
if(top == NULL)
return true; else
return false;
}
```

//Function to insert an element in stack

void push (int value)

```
{
Node *ptr = new Node();
ptr->data = value;
ptr->link = top;
top = ptr;
}
```

//Function to delete an element from the stack

void pop ()

```
{
if ( isempty() )
cout<<"Stack is Empty";
else
{
Node *ptr = top;
top = top -> link;
delete(ptr);
}
}
```

Тема 12 – Линејни динамични АТД

```
// Function to show the element at the top of the stack
```

```
void showTop()
```

```
{
```

```
if ( isempty() )
```

```
    cout<<"Stack is Empty";
```

```
else
```

```
    cout<<"Element at top is : "<< top->data;
```

```
}
```

```
// Function to Display the stack
```

```
void displayStack()
```

```
{
```

```
if ( isempty() )
```

```
    cout<<"Stack is Empty";
```

```
else
```

```
{
```

```
    Node *temp=top;
```

```
    while(temp!=NULL)
```

```
    { cout<<temp->data<<" ";
```

```
      temp=temp->link;
```

```
    }
```

```
    cout<<"\n";
```

```
}
```

```
}
```

```
// Main function
```

```
int main()
```

```
{
```

```
int choice, flag=1, value;
```

```
//Menu Driven Program using Switch
```

```
while( flag == 1) {
```

```
    cout<<"\n1.Push 2.Pop 3.showTop 4.displayStack 5.exit\n";
```

```
    cin>>choice;
```

```
    switch (choice) {
```

```
        case 1: cout<<"Enter Value:\n";
```

```
                cin>>value;
```

```
                push(value);
```

```
                break;
```

```
        case 2: pop();
```

```
                break;
```

```
        case 3: showTop();
```

```
                break;
```

```
        case 4: displayStack();
```

```
                break;
```

```
        case 5: flag = 0;
```

```
                break; } }
```

```
return 0;
```

```
}
```

Тема 12 – Линејни динамични АТД

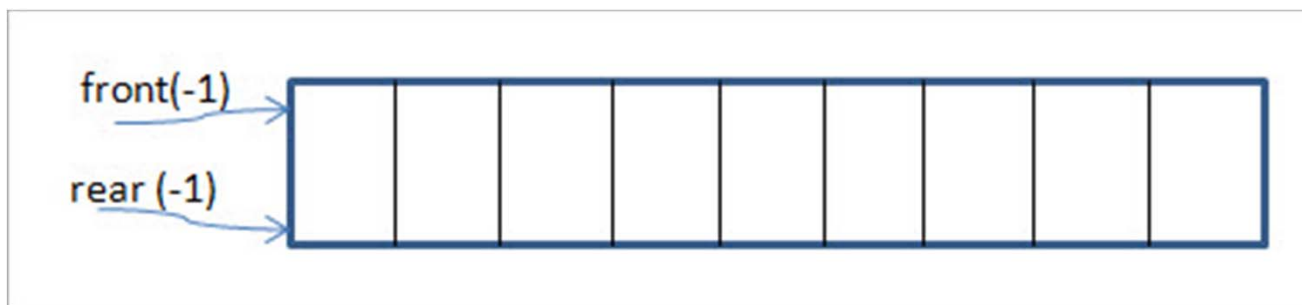
Опашка в C++:

Имаме два края, т.е. „отпред“ и „отзад“ на опашката. Когато опашката е празна, и двата указателя са зададени на -1.

„Задният“ краен указател е мястото, откъдето се вмъкват елементите в опашката. Операцията по добавяне/вмъкване на елементи в опашката се нарича „поставяне на опашка“ („enqueue“).

„Предният“ краен указател е мястото, откъдето елементите се премахват от опашката. Операцията за премахване/изтриване на елементи от опашката се нарича „извеждане от опашка“ („dequeue“).

Когато стойността на задния указател е $\text{size}-1$, тогава казваме, че опашката е пълна. Когато предният указател е нулев, тогава опашката е празна.



Тема 12 – Линејни динамични АТД

Структурата на данните на опашката вклучва следните операции:

- * **EnQueue:** Добавя елемент към опашката. Добавянето на елемент към опашката винаги се извършва в задната част на опашката.
- * **DeQueue:** Премахва елемент от опашката. Елементът се премахва или отстранява от опашката винаги от началото на опашката.
- * **isEmpty:** Проверява дали опашката е празна.
- * **isFull:** Проверява дали опашката е пълна.
- * **peek:** Получава елемент в началото на опашката, без да го премахва.

Тема 12 – Линејни динамични АТД

Добавяне на елемент към опашка - Enqueue

В този процес се изпълняват следните стъпки:

1. Проверка дали опашката е пълна.
2. Ако е пълна, генерира грешка за препълване и ИЗХОД.
3. В противен случай се увеличава „rear“.
4. Добавя се елемент към местоположението, посочено от „rear“.
5. Връщане на успех.

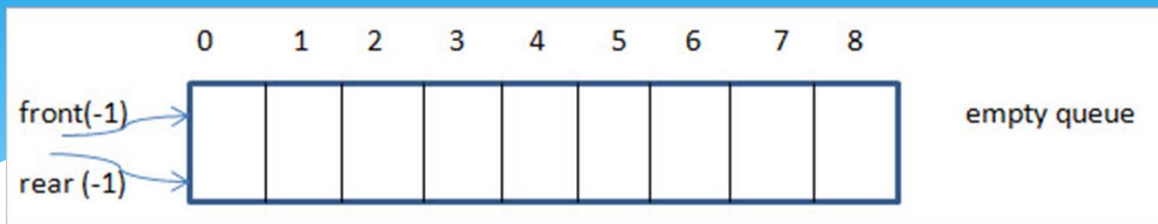
Тема 12 – Линејни динамични АТД

Изваждане на елемент от опашката - Dequeue

Операцијата за премахване на опашката се состои од следните стъпки:

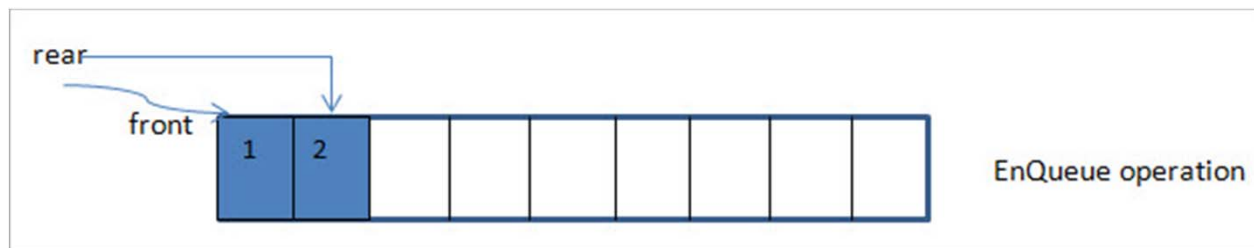
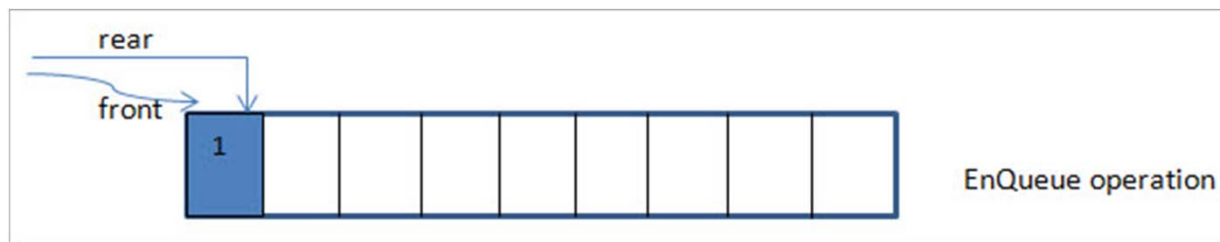
1. Проверка дали опашката е празна.
2. Ако е празна, показва се грешка при преливање и ИЗХОД.
3. В противен случај елементът за достъп е посочен с „front“.
4. Увеличава се „front“, за да посочи следващите достъпни данни.
5. Връщане на успех.

Тема 12 – Линейни динамични АТД



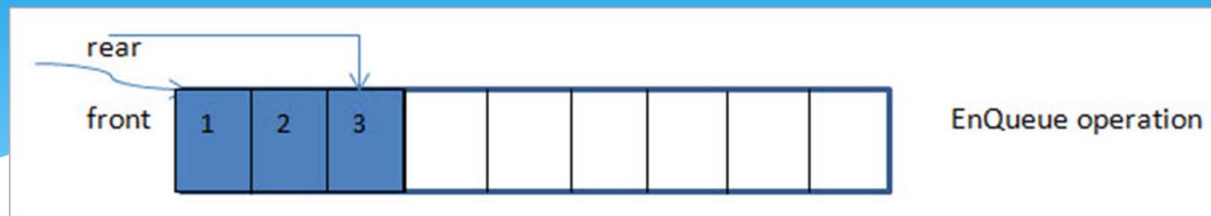
Това е празна опашка и по този начин имаме **rear** и **empty**, зададени на -1.

След това добавяме 1 към опашката и в резултат на това задният указател се премества с едно място напред.



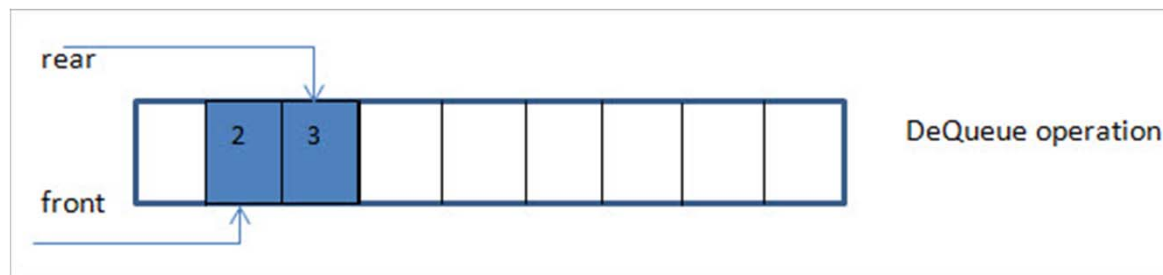
Добавяме елемент 2 към опашката, като преместваме задния указател напред с още една стъпка.

Тема 12 – Линејни динамични АТД



На следващата фигура добавяме елемент 3 и преместваме задния указател с 1.

В този момент задният указател има стойност 2, докато предният указател е на 0-то място.



След това изтриваме елемента, посочен от предния указател. Тъй като предният указател е на 0, елементът, който се изтрива, е 1.

Така първият елемент, въведен в опашката, т.е. 1, се оказва първият елемент, премахнат от опашката. В резултат на това, след първото изваждане от опашката, предният указател сега ще бъде преместен напред до следващото местоположение, което е 1.

Приложение на опашка с масив:

```
#include <iostream>
using namespace std;
#define SIZE 5
int A[SIZE];
int front = -1;
int rear = -1;

//function to check if queue is empty
bool isempty()
{
    if(front == -1 && rear == -1)
        return true;
    else
        return false;
}

//function to insert element in queue
void enqueue ( int value )
{
    if (rear == SIZE-1)
        cout<<"Queue is full \n";
    else
    {
        if( front == -1)
            front = 0;
        rear++;
        A[rear] = value;
    }
}
```

```
//function to remove element from queue
void dequeue ( ) {
    if( isempty() )
        cout<<"Queue is empty\n";
    else
        if( front == rear )
            front = rear = -1;
        else
            front++;
}

//function to display element at front
void showfront( ) {
    if( isempty())
        cout<<"Queue is empty\n";
    else
        cout<<"element at front is:"<<A[front]<<"\n";
}

//function to display elements of the queue
void displayQueue() {
    if(isempty())
        cout<<"Queue is empty\n";
    else
    {
        for( int i=front ; i<= rear ; i++)
            cout<<A[i]<<" ";
        cout<<"\n";
    }
}
```

Приложение на опашка със свързан списък:

// Structure of Node.

```
struct Node {
```

```
int data;
```

```
Node *link;
```

```
};
```

```
Node *front = NULL;
```

```
Node *rear = NULL;
```

//Function to check if queue is empty or not

```
bool isempty()
```

```
{
```

```
if(front == NULL && rear == NULL)
```

```
return true;
```

```
else
```

```
return false;
```

```
}
```

//function to enter elements in queue

```
void enqueue ( int value )
```

```
{
```

```
Node *ptr = new Node();
```

```
ptr->data= value;
```

```
ptr->link = NULL;
```

//If inserting the first element/node

```
if( front == NULL )
```

```
{
```

```
front = ptr;
```

```
rear = ptr; }
```

```
else {
```

```
rear ->link = ptr;
```

```
rear = ptr;
```

```
}
```

```
}
```

Приложение на опашка със свързан списък:

//function to delete/remove element from queue

```
void dequeue ( ) {
```

```
    if( isempty() )
```

```
        cout<<"Queue is empty\n";
```

```
    else
```

```
        //only one element/node in queue.
```

```
        if( front == rear) {
```

```
            free(front);
```

```
            front = rear = NULL; }
```

```
    else {
```

```
        Node *ptr = front;
```

```
        front = front->link;
```

```
        free(ptr);
```

```
    }
```

```
}
```

//function to show the element at front

```
void showfront( ) {
```

```
    if( isempty())
```

```
        cout<<"Queue is empty\n";
```

```
    else
```

```
        cout<<"element at front is:"<<front->data;
```

```
    }
```

//function to display queue

```
void displayQueue() {
```

```
    if (isempty())
```

```
        cout<<"Queue is empty\n";
```

```
    else {
```

```
        Node *ptr = front;
```

```
        while( ptr !=NULL)
```

```
        {
```

```
            cout<<ptr->data<<" ";
```

```
            ptr= ptr->link;
```

```
        } }
```

```
}
```

Тема 12 – Линејни динамични АТД

Свѝрзан списѝк (linked list)

В програмирањето, свѝрзан списѝк е линејна колекција од елементи од данни, чийто ред не е даден од тяхното физическо разположение в паметта.

Вместо това всеки елемент сочи към следващия.

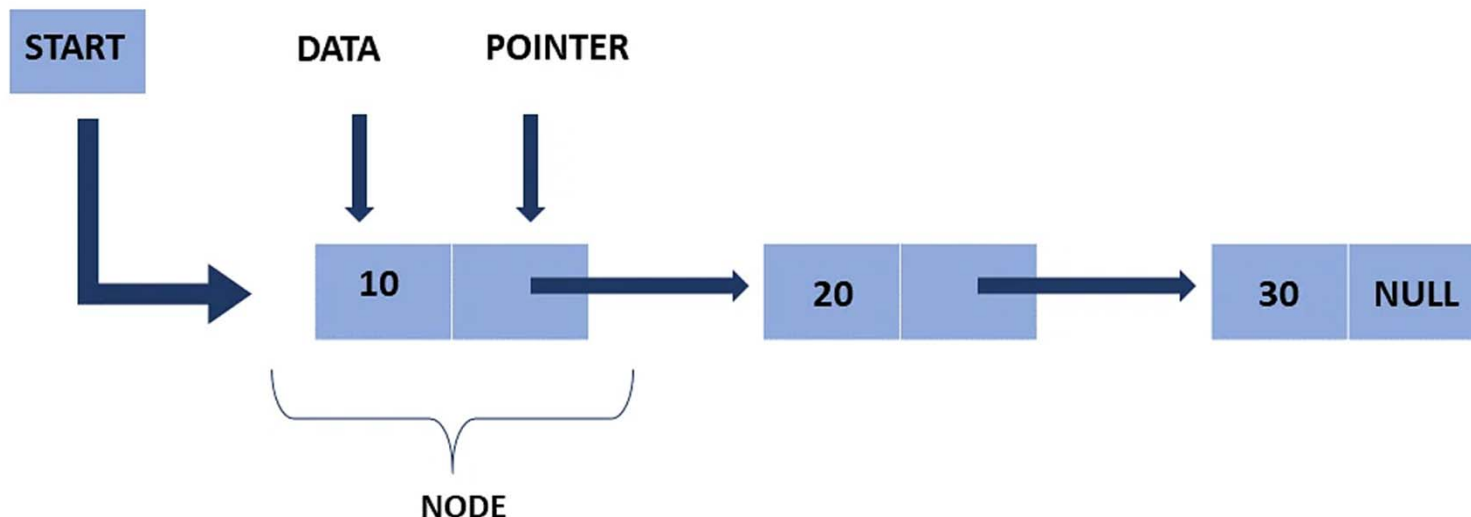
Това е структура от данни, състояща се от колекция от възли, които заедно представляват последователност. В най-основната си форма всеки възел съдържа: **данни и препратка** (с други думи, връзка, указател) към следващия възел в последователността.

Тази структура позволява ефективно вмѝкване или премахване на елементи от всяка позиция в последователността по време. По-сложните варианти на свѝрзания списѝк добавят допълнителни връзки, позволяващи по-ефективно вмѝкване или премахване на възли на произволни позиции.

Недостатѝкът на свѝрзаните списѝци е, че времето за достѝп е линејно, тоест по-бѝрз достѝп, като произволен достѝп, не е осѝществим. Масивите имат по-добро местоположение на кеша в сравнение със свѝрзаните списѝци.

Тема 12 – Линейни динамични АТД

Свързаният списък се състои от възли, като всеки възел се състои от две полета. Първото поле се състои от данни, а второто поле се състои от указатели, които сочат към друг възел.



Тема 12 – Линејни динамични АТД

Създаване на възел и деклариране на свързани списъци:

```
struct node
{
    int data;
    struct node * next;
};
struct node * n;
n=(struct node*)malloc(sizeof(struct node*));
```

Това е декларация на възел, който се състои от първата променлива като данни и следващата като указател, който ще запази адреса на следващия възел. Тук трябва да използвате функцията **malloc**, за да разпределите динамично памет за възлите.

Свързаният списък разчита до голяма степен на функцията **malloc()** за **динамично** разпределяне на памет за нови възли.